

Experimental Protocol & Runbook — MMseqs2-Triton

Companion to: [01-research-paper-plan.md](#) **Audience:** the operator running experiments on the GPU cluster. **Purpose:** turn Section 5 of the plan into an executable checklist — environment, datasets, exact commands, ablation sweeps, correctness gates, and empty results tables to fill in. **No results are recorded here yet;** every results table is a template.

Conventions. `$NV` = an NVIDIA node, `$AMD` = an AMD node. Replace `${...}` placeholders with site-specific paths. Commands are illustrative and follow the invocations reported in Kallenborn et al. (2025); confirm exact flags against the installed MMseqs2 commit before a production run. Treat every "Gate:" line as a hard stop — do not proceed to the next phase until it passes.

0. Pre-flight checklist

- ☐ Reserve nodes: ≥1 NVIDIA L40S (primary), plus H100 and A100 if available; ≥1 AMD MI300X.
- ☐ Confirm fixed GPU clocks (disable boost variability) for benchmarking: `nvidia-smi -lgc <clk> / rocm-smi --setperflevel high`.
- ☐ Confirm host: 2× AMD EPYC class CPU, ≥1 TB RAM, fast NVMe scratch (for the 30 M-sequence DB and streaming tests).
- ☐ Provision ≥2 TB scratch for databases and intermediate indices.
- ☐ Record exact versions in `env_manifest.txt` (template in §1.3).

1. Environment

1.1 Software stack (pin these)

Component	NVIDIA target	AMD target
GPU runtime	CUDA 12.x	ROCm 6.x
Triton	≥ 3.x	≥ 3.x (ROCm build)
PyTorch	≥ 2.4 (cu12x)	≥ 2.4 (rocm6x)
MMseqs2	GPU-enabled commit (record hash)	CPU build (record hash)
Profilers	Nsight Compute + Nsight Systems	rocprof + omniperf
Baselines	BLAST 2.16.0, DIAMOND 2.1.9, HMMER 3.4	same (CPU)

1.2 Build / install (sketch)

```
# MMseqs2 (GPU build for NVIDIA nodes)
git clone https://github.com/soedinglab/MMseqs2.git && cd MMseqs2
# record commit:
git rev-parse HEAD | tee ${RESULTS}/mmseqs_commit.txt
cmake -DHAVE_CUDA=1 -DCMAKE_BUILD_TYPE=Release -B build && cmake --build
```

```
build -j

# Triton kernels (this project) – same source, both backends
pip install -e ./mmseqs2-triton          # NVIDIA node (cu12x wheels)
pip install -e ./mmseqs2-triton          # AMD node   (rocm6x wheels)

# Baselines
# BLAST/PSI-BLAST 2.16.0, DIAMOND 2.1.9, HMMER 3.4 (jackhmmer/phmmer)
```

1.3 `env_manifest.txt` template (fill once per node)

node_id: _____

vendor/gpu: _____ (e.g., NVIDIA L40S 48GB / AMD MI300X 192GB)

driver: _____

cuda_or_rocm: _____

triton_version: _____

pytorch_version: _____

mmseqs_commit: _____

cpu / sockets / RAM: _____

fixed_clock_MHz: _____

date: _____

2. Datasets (acquire and verify once)

ID	Dataset	Use	Size / notes	Acquired?
D1	MMseqs2 SCOP/UniProt benchmark (6,370 queries; 3.4 M + 27 M reversed = 30,430,281 refs)	Sensitivity (ROC1), real-sequence TCUPS	from the MMseqs2 benchmark tarball	☐
D2	Synthetic uniform-length sequences (lengths 32–2048; 5 M refs per length)	Best-case TCUPS	generated by script <code>gen_synth.py</code>	☐
D3	Replicated D1 at 1×/4×/16×	DB scaling & streaming	derived from D1	☐
D4	CASP14 free-modeling targets (20 sequences)	End-to-end ColabFold TM-score	from CASP14	☐
D5	AFDB50 sample (6,370 structures as 3Di) + AFDB50 v4 DB	Foldseek transfer	from AFDB / Foldseek analysis repo	☐
D6	UniRef90 (e.g., 2022_01, ~144 M)	Optional Colab-style streaming test	large; streaming path	☐

- ☐ Verify D1 reference count == **30,430,281**.

- ☐ Store SHA256 of each downloaded archive in `datasets/CHECKSUMS.txt`.
- ☐ Build MMseqs2 DBs/indices once and reuse: `mmseqs createdb`, `mmseqs createindex` (record index build time, e.g., for the 30 M set).

3. Baseline reference runs (Phase 0)

Establish the numbers the Triton kernels must match/beat. Use matched sensitivity where possible (target ROC1 $\approx$ 0.40).

3.1 Sensitivity baselines (D1)

```
# MMseqs2-CPU k-mer
mmseqs easy-search queries.fasta refDB res_kmer.m8 tmp \
  -s 8.5 -e 10000 --max-seqs 4000 --threads 128

# MMseqs2-CPU gapless (ungapped prefilter)
mmseqs easy-search queries.fasta refDB res_gapless.m8 tmp \
  --prefilter-mode 1 -e 10000 --max-seqs 4000 --threads 128

# MMseqs2-GPU (CUDA) – NVIDIA node, primary target
mmseqs easy-search queries.fasta refDB res_gpu.m8 tmp \
  --prefilter-mode 1 --gpu 1 --gpu-server 1 -e 10000 --max-seqs 4000 --
threads 1

# Context: BLAST / PSI-BLAST / DIAMOND ultra / JackHMMER (see plan §5.2)
```

Gate (Phase 0): reproduce published ROC1 within reporting precision.

Tool	Mode	ROC1 1-it	2-it	3-it	Reproduced?
MMseqs2-CPU	-s 8.5	(exp. 0.391)	(0.606)	(0.665)	☐
MMseqs2-CPU	gapless	(0.40)	(0.612)	(0.669)	☐
MMseqs2-GPU	gapless (CUDA)	(0.40)	(0.612)	(0.669)	☐
BLAST	—	(0.401)	—	—	☐
PSI-BLAST	—	—	(0.574)	(0.591)	☐
JackHMMER	—	(0.40)	(0.614)	(0.685)	☐
DIAMOND	ultra-sens	(0.409)	—	—	☐

(Values in parentheses are the published reference points to reproduce, not new results.)

3.2 Profiling baseline

```
# NVIDIA
ncu --set full --target-processes all -o prof_gpu_cuda \
```

```
mmseqs ungappedprefilter queryDB refDB out tmp --gpu 1
nsys profile -o nsys_gpu_cuda mmseqs ...

# AMD (CPU baseline + later Triton)
rocpfrof --hip-trace --stats mmseqs ungappedprefilter ... # AMD CPU/Triton
path
```

Deliverable: bottleneck table (per-kernel time %, calls, occupancy, regs/thread, achieved HBM BW, L2 hit %, divergence) + roofline plot. Template in §8.

4. Phase-by-phase experiments

Phase 1 — Fused gapless filter on NVIDIA (H1)

Run. Swap in the Triton filter (e.g., `--prefilter-backend triton`) and run D2 (synthetic TCUPS) and D1 (real TCUPS).

```
# Synthetic TCUPS sweep (D2): for each length l in 32 64 128 256 512 1024
2048
python bench_tcups.py --backend {cuda,triton} --length $l \
  --num-refs 5000000 --gpu $NV --dtype bf16 --repeats 5 --warmup 20

# Real TCUPS (D1)
python bench_tcups.py --backend {cuda,triton} --db refDB_30M \
  --queries queries.fasta --gpu $NV --repeats 5 --warmup 20
```

Correctness gate (bit-exact).

```
python verify_scores.py --ref cuda --test triton --pairs
correctness_set.tsv
# REQUIRE: max_abs_diff == 0 for filter scores (fp32/int path)
# bf16 arm: REQUIRE identical ROC1 on D1 or document any deviation
```

Gate (Phase 1): `max_abs_diff == 0` AND Triton TCUPS $\geq 0.85 \times$ CUDA at every length.

Results template — synthetic TCUPS (D2), NVIDIA L40S:

Length	CUDA TCUPS	Triton TCUPS	Ratio (T/C)	% of peak (Triton)	Scores exact?
32					☐
64					☐
128					☐
256					☐
512					☐

Length	CUDA TCUPS	Triton TCUPS	Ratio (T/C)	% of peak (Triton)	Scores exact?
1024					☐
2048					☐

Phase 4a — Port filter to AMD MI300X (H2, headline)

Run the **same Triton source, no algorithm changes** (only autotune configs may differ):

```
# On $AMD (ROCm). Same kernel module, recompiled for AMDGCN.
python bench_tcups.py --backend triton --length $l --num-refs 5000000 \
  --gpu $AMD --dtype bf16 --repeats 5 --warmup 20
# AMD CPU baseline for the speedup ratio:
mmseqs easy-search queries.fasta refDB res_amd_cpu.m8 tmp \
  --prefilter-mode 1 -e 10000 --max-seqs 4000 --threads <AMD_cores>
```

Gate (Phase 4a / H2): Triton filter runs on MI300X unmodified AND $\geq 10\times$ the AMD CPU gapless baseline AND scores bit-exact.

Results template — cross-vendor filter:

Metric	NVIDIA L40S (Triton)	AMD MI300X (Triton)	AMD CPU (gapless)	T(AMD)/CPU ratio
Real-seq TCUPS (D1)				
% of per-vendor peak			n/a	
Achieved HBM BW (GB/s)			n/a	
Scores bit-exact vs CUDA	☐	☐	☐	
Same Triton source (no edits)?	☐	☐	n/a	

Phase 3 — Fusion increments & launch accounting (H3)

Run the **fusion ablation**: progressively enable fusions and record launches + bandwidth + TCUPS.

```
for fuse in none mask mask+select mask+select+handoff; do
  python bench_fusion.py --fusion $fuse --gpu $NV --db refDB_30M --repeats
  5
  # capture launch count via nsys/rocpfrof kernel trace
done
```

Gate (Phase 3): $\geq 50\%$ launch reduction (fused vs unfused Triton) AND achieved HBM BW $\geq 50\%$ of peak on both vendors.

Results template — fusion ladder (per vendor):

Fusion level	#Kernel launches	% reduction vs unfused	HBM BW (GB/s)	% peak	TCUPS
Unfused		— (ref)			
+ tantan mask					
+ threshold/compaction					
+ cross-stage handoff					
CUDA MMseqs2-GPU (ref)					

Phase 2 — Triton SWG kernel + decision gate

```
python bench_swg.py --backend {cudasw4,triton} --candidates topk.idx \
  --gpu $NV --group-size {4,8,16,32} --repeats 5
python verify_swg.py --ref cudasw4 --test triton # identical optimal scores
```

Decision gate: if Triton SWG $< 0.7 \times$ CUDASW++4.0 throughput after autotuning → **enable hybrid dispatcher** (Triton filter everywhere; CUDA SWG on NVIDIA; Triton SWG on AMD). Record the decision and the measured ratio either way.

SWG backend	Group size	Throughput	Ratio vs CUDASW4	Optimal scores identical?
CUDASW++4.0	best	(ref)	1.00	☐
Triton	4/8/16/32			☐

Decision: pure-Triton / hybrid

Phase 5 — End-to-end & transfer (H4)

ColabFold MSA (D4, 20 CASP14 targets):

```
# Configs: AF2(JackHMMER+HHblits) | ColabFold-MMseqs2-CPU | ColabFold-MMseqs2-GPU(CUDA) | ColabFold-MMseqs2-Triton
# Omit template search + relaxation; 3 recycles; 5 AF2 models on one GPU.
colabfold_batch --msa-backend mmseqs2-triton casp14.fasta out_triton/
python score_tm.py --pred out_triton/ --native casp14_native/ # TM-score
```

Config	MSA wall time	Total wall time	TM-score (mean±sd)
AlphaFold2 (JackHMMER+HHblits)			
ColabFold MMseqs2-CPU			
ColabFold MMseqs2-GPU (CUDA)			
ColabFold MMseqs2-Triton			

Foldseek transfer (D5):

```
foldseek easy-search queries_3di refDB_afdb50 aln tmp \
--prefilter-backend triton --gpu 1 -e 10 --max-seqs 2000
```

Backend	Speed (queries/s)	SCOPE fam	superfam	fold
Foldseek-CPU k-mer		(≈0.861)	(≈0.487)	(≈0.106)
Foldseek-GPU (CUDA)		(≈0.874)	(≈0.493)	(≈0.108)

Foldseek-Triton

Gate (Phase 5 / H4): TM-score 95% CI includes the CUDA config's mean; Foldseek sensitivity within seed variance.

Phase 6 — Economics, scaling, tail latency

```
# Multi-GPU & streaming
for g in 1 2 4 8; do python bench_scale.py --gpus $g --db
refDB_{1x,4x,16x}; done
# Energy: sample power during single-batch run
nvidia-smi --query-gpu=power.draw --format=csv -lms 100 > power_nv.csv #
NVIDIA
rocm-smi --showpower -i 0 --csv > power_amd.csv #
AMD
# Tail latency (single-query / server mode), 10,000 trials
python bench_latency.py --backend triton --trials 10000 --batch 32
```

Scenario	p50 (ms)	p95	p99	max	\$/query	J/query
MMseqs2-GPU CUDA (NV)						
MMseqs2-Triton (NV)						
MMseqs2-Triton (AMD)						
DB scaling	1 GPU	2	4	8	streamed/in-mem ratio	
TCUPS (1×)						

DB scaling	1 GPU	2	4	8	streamed/in-mem ratio
TCUPS (4×)					
TCUPS (16×, streamed)					

5. Full ablation sweep (master grid)

Run on **both** vendors unless noted. Record into [results/ablations.csv](#).

Axis	Settings	Primary metric
Precision	fp32, bf16 , fp16	TCUPS, ROC1, NaN count
Fusion level	none / +mask / +select / +handoff	launches, BW, TCUPS
BLOCK_J	autotune set	TCUPS
cells/thread	autotune set	TCUPS
thread-group size	4, 8, 16 (filter); 4–32 (SWG)	TCUPS
num_warps / num_stages	autotune set	TCUPS, occupancy
sequence length	32–2048, >2048 tiled	TCUPS, % peak
GPUs	1, 2, 4, 8	scaling efficiency
DB residency	in-memory vs streamed	TCUPS ratio
vendor	NVIDIA vs AMD	all of the above

Config transferability check: record whether the NVIDIA-optimal autotune config is also AMD-optimal (Y/N) per length bucket.

6. Correctness gates (must pass before any deployment)

Gate	Requirement	Tool	Pass?
Filter bit-exact	max abs(Δ score) == 0 vs CPU/CUDA (fp32/int path)	verify_scores.py	☐
bf16 sensitivity-neutral	ROC1 identical on D1 (or documented)	sensitivity harness	☐
SWG optimal-score identity	identical optimal scores vs CUDASW++4.0	verify_swg.py	☐
End-to-end ROC1	matches MMseqs2-GPU to reporting precision	benchmark	☐
Determinism	repeated runs identical (fixed seed)	rerun ×3	☐

7. Statistical analysis recipe

- **Throughput/latency:** median of ≥5 runs, ≥20 warmup; report min/median/max; fixed clocks.
- **Quality deltas (TM-score, Foldseek sens., stochastic ROC1):** 5 seeds {0,1,2,3,4}; **paired bootstrap 95% CI** (10,000 resamples) on per-seed deltas vs the CUDA config; **paired Wilcoxon** p-value. Report CI and p.
- **Decision rule:** "unchanged" ⇔ 95% CI of the delta includes 0.
- Script: `stats.py --metric tmscore --a triton --b cuda --seeds 0 1 2 3 4 --bootstrap 10000`.

8. Profiling/roofline data-collection template

Metric	NVIDIA L40S	NVIDIA H100	NVIDIA A100	AMD MI300X
Achieved occupancy (%)				
Registers / thread				
Shared mem / block (KB)				
Achieved HBM BW (GB/s)				
% of peak BW				
L2 hit rate (%)				
Warp/wavefront divergence (%)				
Arithmetic intensity (FLOP/byte)				
Theoretical peak (TCUPS, per-vendor TPP)				
Achieved TCUPS				
Efficiency (% of peak)				

Per-vendor TPP reminder: $TPP = (\text{\#SMs_or_CUs} \times \text{throughput_per_instruction} \times \text{clock}) / \text{cycles_per_cell_update}$. Fill the architectural constants per GPU before computing efficiency.

9. Results-to-manuscript handoff

When all gates pass and tables are filled:

1. Copy confirmed numbers into `01-research-paper-plan.md`, replacing each "(to be measured, H#)" with the value + CI.
2. Mark each hypothesis H1–H4 as **supported** / **partially supported** / **refuted** with the deciding metric.
3. Generate figures: TCUPS-vs-length (per vendor), fusion ladder, roofline, cross-vendor bar, end-to-end wall-time stack, TM-score parity.
4. Record final artifact commit + autotune caches for reproducibility.

End of `02-experimental-protocol.md`.